

Algoritmi e Strutture Dati – Parte A – 22/08/2019

Esercizio A1 – Punti ≥ 8

Durante un orale di Algoritmi, non ricordando come è implementata la funzione `Merge()` contenuta in `MergeSort()`, uno studente ha deciso di improvvisare e mi ha proposto questa versione:

```
Merge(ITEM A[], int primo, int ultimo, int mezzo)
```

```

for i = primo + 1 to ultimo do
    ITEM temp = A[i]
    int j = i
    while j > 1 and A[j - 1] > temp do
        A[j] = A[j - 1]
        j = j - 1
    A[j] = temp

```

Scrivere le equazioni di ricorrenza corrispondenti al costo computazionale nel caso *ottimo* e al caso *pessimo* dell'algoritmo `MergeSort()` così modificato e risolverle tramite Master Theorem.

Esercizio A2 – Punti ≥ 10

Sia T un albero binario di ricerca i cui nodi hanno un campo *color* che può assumere il valore RED o BLACK. Scrivere un algoritmo

boolean isRedBlack(TREE T)

che prenda in input un albero T e restituisca **true** se e solo se la colorazione rispetta le regole red-black, **false** altrimenti.

Discutere informalmente la correttezza dell'algoritmo e calcolare la sua complessità computazionale.

Esercizio A3 – Punti ≥ 12

In un corso di laurea sono previsti n esami, tutti obbligatori. Esistono m vincoli di propedeuticità, rappresentati tramite un DAG $G = (V, E)$. Se $(u, v) \in E$, l'esame u è propedeutico all'esame v ; questo significa che v deve essere sostenuto in una sessione d'esame successiva a quella in cui si è svolto u . Se due o più esami non hanno vincoli di propedeuticità, allora possono essere sostenuti tutti all'interno di una stessa sessione.

Scrivere un algoritmo

int minSessions(GRAPH G)

che prenda in input un grafo di propedeuticità G e restituisca il numero minimo di sessioni necessarie per completare tutti gli esami.

Discutere informalmente la correttezza dell'algoritmo e calcolare la sua complessità computazionale.

Ad esempio, nel grafo seguente, A , B possono essere dati nella prima sessione, C nella seconda, D nella terza. L'algoritmo deve quindi restituire il valore 3.

